Fundamentos de Recuperación de Información

El problema de búsqueda y el procesamiento de texto

Iván Carrera

Escuela Politécnica Nacional
Recuperación de Información

Lo que podremos explicar al terminar

1. Distinguir necesidad de información, consulta y relevancia a partir de una situación de búsqueda.
2. Explicar el papel de documentos, colección, índice y resultados dentro de un sistema de RI.
3. Transformar un texto en tokens y términos, justificando las reglas elegidas.
4. Analizar qué coincidencias ganamos y qué distinciones perdemos con cada transformación.

Pregunta de la clase: ¿cómo preparamos una colección para encontrar información útil?

Un problema que comienza antes de escribir la consulta

Una estudiante necesita evidencia para reducir la deserción de primer año en su institución.

Su consulta expresa solo parte de esa necesidad.

¿Puede un resultado ser relevante aunque utilice otras palabras?



La RI busca material que satisfaga una necesidad

La **Recuperación de Información (RI)** estudia cómo encontrar, dentro de una colección, material que ayude a satisfacer una necesidad de información.

El material suele ser texto, pero también puede incluir imágenes, audio u otras modalidades. Inicialmente trabajaremos con documentos textuales.

Problema central

El sistema observa la consulta y la representación de los documentos. La necesidad completa del usuario puede quedar parcialmente implícita.

Una coincidencia de palabras aporta evidencia de relevancia, pero su utilidad depende del significado y del propósito de la búsqueda.

Fuente conceptual: Manning, Raghavan y Schütze, cap. 1, introducción y sec. 1.1.

Necesidad, consulta y relevancia son conceptos diferentes

Concepto	Significado	Ejemplo
Necesidad de información	Lo que el usuario desea conocer para resolver su problema.	Evidencia sobre estrategias para reducir la deserción de primer año.
Consulta	Expresión que comunica parte de esa necesidad al sistema.	`abandono universidad primer año`
Relevancia	Relación entre el contenido de un documento y la necesidad del usuario.	Un estudio sobre tutorías y permanencia puede aportar evidencia útil.

Una misma necesidad admite varias consultas. Una misma consulta puede expresar necesidades distintas.

Fuente conceptual: Manning et al., sec. 1.1 y 8.1.

Una consulta breve puede ser ambigua



Un documento puede coincidir con la consulta y aportar poco a la necesidad concreta.

Fuente conceptual: Manning et al., sec. 8.1. Ejemplo adaptado.

La relevancia requiere un criterio explícito

Para nuestro caso, consideraremos si el documento aborda una intervención, una población y un resultado relacionados con la permanencia universitaria.

Documento	Juicio para esta necesidad
Evaluación de tutorías en primer año universitario	Relevante: aporta evidencia de una intervención.
Estadísticas generales de matrícula universitaria	Parcialmente relevante: proporciona contexto.
Abandono de mascotas	No relevante: corresponde a otro tema.

Los juicios deben justificarse. Pueden ser binarios o graduados, según el diseño de la evaluación.

Fuente conceptual: Manning et al., sec. 8.1. Criterio y ejemplos de elaboración docente.

Documento y archivo no siempre coinciden

Un **documento** es la unidad que el sistema representa y recupera. Un **archivo** es una unidad de almacenamiento.

Archivo disponible	Posible unidad de recuperación
Un CSV con mil noticias	Cada noticia puede ser un documento.
Un PDF con un libro completo	Cada capítulo puede ser una unidad.
Un artículo científico	El artículo completo o sus secciones.

La granularidad cambia el contexto disponible y la facilidad para localizar la información. En esta clase mantendremos documentos cortos completos.

Fuente conceptual: Manning et al., sec. 2.1.2.

La colección define el universo de búsqueda

Una **colección o corpus** reúne las unidades documentales disponibles para el sistema.

Cada documento necesita un ID estable y un texto. Los metadatos describen su fuente, idioma o fecha.

Solo podemos recuperar material que haya ingresado a la colección.



Fuente conceptual: Manning et al., sec. 1.1. Croft, Metzler y Strohman, sec. 2.2.

Búsqueda en datos estructurados y en texto

Aspecto	Consulta estructurada	Recuperación textual
Ejemplo	Matrículas del período 2026-B.	Estudios sobre permanencia estudiantil.
Criterio de selección	Condición definida sobre un campo.	Evidencia de relación con la necesidad.
Salida frecuente	Registros que satisfacen la condición.	Conjunto o ranking de documentos.
Dificultad característica	Calidad y consistencia de campos.	Ambigüedad y variación del lenguaje.

Un buscador puede combinar filtros exactos por fecha o idioma con recuperación por contenido textual.

Fuente conceptual: Manning et al., cap. 1, introducción.

El repaso de ML aporta herramientas para RI

Concepto revisado	Relación con recuperación
Instancia y features	Representar documentos y consultas mediante características.
Clasificación	Asignar documentos a categorías predefinidas.
Clustering	Agrupar documentos sin categorías dadas.
Generalización	Comprobar si un método funciona con nuevas consultas.

La tarea de búsqueda que estudiaremos recibe una consulta y devuelve documentos relacionados con ella. Las categorías de clasificación no sustituyen los juicios de relevancia para cada necesidad.

Fuente conceptual: Manning et al., cap. 1, introducción. Conexiones con el repaso de ML.

La arquitectura separa preparación y búsqueda



La preparación construye o actualiza el índice. Cada consulta utiliza ese índice para recuperar resultados.

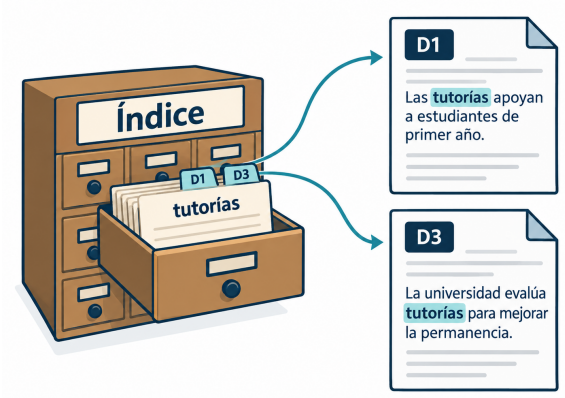
Fuente conceptual: Croft et al., sec. 2.2.

El índice permite localizar candidatos con rapidez

Un **índice** organiza información derivada de los documentos para evitar recorrer todo el texto en cada búsqueda.

El término `tutorías` permite localizar D1 y D3.

Posteriormente construiremos el índice invertido y estudiaremos consultas booleanas.



El índice localiza; el texto aporta contexto

Fuente conceptual: Croft et al., sec. 2.2.

Un ranking expresa una estimación de relevancia

En recuperación ordenada, el sistema asigna un score y presenta primero los documentos con mayor valor según su modelo.

$$\text{score}(q, d) \qquad d \in \mathcal{D}$$

Aquí q es la consulta, d un documento y $\mathcal{D}$ la colección. *Top-k* designa los primeros k resultados.

Interpretación

El score es una estimación del modelo. Su valor no equivale necesariamente a una probabilidad y una primera posición no garantiza utilidad.

El ranking clásico y sus fórmulas corresponden a temas futuros. Hoy identificamos su función en el sistema.

Fuente conceptual: Manning et al., sec. 1.4. Croft et al., sec. 2.2.

Buscar es un proceso que puede repetirse

El usuario examina resultados, aprende vocabulario y puede reformular la consulta.



Fuente conceptual: Croft et al., sec. 2.2, interacción con el usuario.

Un buscador necesita calidad y eficiencia

Efectividad
¿Los resultados ayudan a satisfacer la necesidad?

Depende de la colección, la representación, el modelo y los criterios de relevancia.

Eficiencia
¿Cuánto tiempo y recursos requiere la búsqueda?

Depende de los índices, los algoritmos y el tamaño de la colección.

Para discutir

Un sistema responde en 20 ms, pero devuelve documentos de otro tema. ¿Qué propiedad está fallando?

Mediremos la calidad de la recuperación de forma sistemática posteriormente.

Fuente conceptual: Croft et al., sec. 2.1.

Procesar texto significa elegir una representación

El texto original contiene diferencias de escritura, flexión, puntuación y formato. Un sistema léxico debe decidir cuáles conservar.

Operación	Decisión principal
Tokenización	¿Dónde comienzan y terminan las unidades?
Normalización	¿Qué variantes consideramos equivalentes?
Stopwords	¿Qué tokens excluimos de esta representación?
Stemming o lematización	¿Cómo agrupamos formas morfológicas?

Estas operaciones forman una política de procesamiento. Su combinación y orden dependen del sistema.

Fuente conceptual: Manning et al., sec. 2.2.

Nuestro corpus de ejemplo

ID	Texto
D1	Las tutorías apoyan a estudiantes de primer año.
D2	Un estudiante abandona la universidad por dificultades económicas.
D3	La universidad evalúa tutorías para mejorar la permanencia.
D4	El estudio analiza el abandono de mascotas.

Consulta para los ejemplos de coincidencia: `tutorías estudiantes`

Mantendremos los IDs y el texto original. Las versiones procesadas serán representaciones adicionales.

Para discutir

¿Qué documentos esperamos relacionar con la necesidad de reducir la deserción universitaria?

Antes de tokenizar hay que obtener el texto

La extracción convierte el contenido de un formato a una secuencia de caracteres utilizable.

Formato	Problema que debemos revisar
HTML	Separar contenido de menús y etiquetas.
PDF digital	Revisar orden de lectura, columnas y saltos de línea.
Documento escaneado	Reconocer texto con OCR y revisar errores.
CSV	Elegir qué campos contienen texto y qué fila define un documento.

Un error de extracción puede producir términos falsos o eliminar información antes de la búsqueda.

Fuente conceptual: Manning et al., sec. 2.1.1.

La tokenización divide el texto en unidades

Tokenizar es segmentar una secuencia de caracteres en unidades de procesamiento llamadas tokens.

Texto D1
Las tutorías apoyan a estudiantes de primer año.

Salida bajo una regla explícita
`[Las, tutorías, apoyan, a, estudiantes, de, primer, año]`

En esta regla, los espacios delimitan palabras y el punto final no forma parte de un token.

Consecuencia

La regla decide qué unidades podrá reconocer después el sistema. Dividir solo por espacios constituye un punto de partida, pero requiere revisar casos especiales.

Fuente conceptual: Manning et al., sec. 2.2.1.

Token, tipo y término no son intercambiables

Texto: `datos de datos`
Regla: separar por espacios y excluir de como stopword.

Concepto	Qué contamos	Resultado
Token	Cada aparición de una unidad en el texto.	3 apariciones
Tipo	Cada forma distinta antes del filtrado.	2: datos, de
Término	Forma que conservamos en el vocabulario del sistema.	1: datos

El número de tokens mide apariciones. El número de tipos o términos distintos mide diversidad bajo una política determinada.

Fuente conceptual: Manning et al., sec. 2.2.1.

Los límites de un token dependen del dominio

Texto	Decisión posible	Riesgo de una regla indiscriminada
`C++`	Conservar como unidad.	Quitar signos lo confunde con `C`.
`COVID-19`	Conservar o dividir con una regla definida.	Cambiar la forma puede impedir coincidencias.
`3,14`	Reconocer como número decimal según el contexto.	Dividir por puntuación genera dos números.
`Nueva York`	Dos tokens, con posibilidad de conservar la frase.	Palabras separadas no aseguran la entidad.

Para discutir

¿Qué regla usaríamos para buscar ofertas laborales que exigen C++?

Fuente conceptual: Manning et al., sec. 2.2.1. Ejemplos adaptados.

Normalizar permite reunir variantes de escritura

La **normalización** asigna una forma común a variantes que decidimos tratar como equivalentes.

Entrada	Salida posible	Regla
`Tutorías`	`tutorías`	Conversión a minúsculas.
`tutorías`	`tutorias`	Eliminación de tildes vocálicas.
`E.P.N.`	`epn`	Regla específica para esta sigla.

Cada regla amplía algunas coincidencias y puede fusionar formas que convendría distinguir. La normalización léxica de esta clase es distinta de la normalización de vectores.

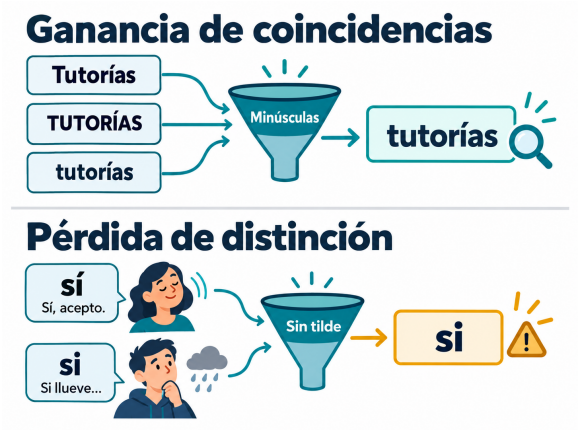
Fuente conceptual: Manning et al., sec. 2.2.3.

Normalizar también puede borrar diferencias útiles

Mayúsculas
`US` puede ser una sigla. `us` es un pronombre en inglés. Convertir ambas a minúsculas las fusiona.

Tildes
`sí` y `si` tienen funciones distintas. Quitar la tilde pierde esa distinción.

El tratamiento de `ñ` requiere una decisión propia: `año` y `ano` tienen significados diferentes.



Fuente conceptual: Manning et al., sec. 2.2.3. Ejemplos en español de elaboración docente.

Las stopwords dependen de la política de búsqueda

Una **stopword** es una forma que el sistema decide excluir de su vocabulario, frecuentemente por su alta frecuencia y baja capacidad de discriminar documentos.

Lista de ejemplo: `{las, a, de}`.

D1 después de minúsculas y filtrado
`[tutorías, apoyan, estudiantes, primer, año]`

La eliminación puede reducir el tamaño de la representación. Su conveniencia depende del corpus, el dominio y las consultas.

Una decisión que requiere evidencia

Frecuencia alta no equivale a ausencia de significado. Conservar palabras frecuentes también es una política válida.

Fuente conceptual: Manning et al., sec. 2.2.2.

Una palabra frecuente puede cambiar la intención

Texto o consulta	Efecto de eliminar palabras
`vuelos a Quito` frente a `vuelos desde Quito`	Perder las preposiciones puede borrar la dirección del viaje.
`no requiere experiencia`	Perder `no` elimina una negación decisiva.
`El nombre de la rosa`	Eliminar palabras altera la representación de un título.

Para discutir

¿Usaríamos la misma lista de stopwords para noticias generales y para requisitos de ofertas laborales?

Las consultas por frases y las tareas sensibles a negación requieren especial cuidado.

Fuente conceptual: Manning et al., sec. 2.2.2. Ejemplos adaptados.

El stemming agrupa formas mediante reglas

El **stemming** aplica reglas heurísticas, usualmente sobre afijos, para obtener una base común. La salida puede no ser una palabra del diccionario.

Entrada en inglés	Salida del stemmer Porter
`connected`	`connect`
`connecting`	`connect`
`connections`	`connect`

Los algoritmos son específicos del idioma. Diferentes stemmers pueden formar grupos diferentes, por lo que debemos identificar el algoritmo utilizado.

Fuente conceptual: Manning et al., sec. 2.2.4, ejemplo Porter.

La lematización busca la forma de diccionario

La **lematización** utiliza conocimiento léxico y morfológico para asociar una forma con su lema. Algunos analizadores también utilizan el contexto.

Forma	Lema esperado	Interpretación
`estudiantes`	`estudiante`	Sustantivo plural.
`apoyan`	`apoyar`	Forma verbal.
`fui`	`ir` o `ser`	El contexto distingue los usos.

"Fui a clase" y "fui tutor" ilustran la ambigüedad. Las salidas concretas dependen del analizador y deben revisarse.

Fuente conceptual: Manning et al., sec. 2.2.4. Ejemplos en español de elaboración docente.

Stemming y lematización resuelven problemas relacionados

Aspecto	Stemming	Lematización
Mecanismo	Reglas heurísticas sobre formas.	Léxico y análisis morfológico.
Resultado	Una base que puede no ser palabra.	Una forma de diccionario.
Contexto	Generalmente no lo utiliza.	Puede utilizarlo según el analizador.
Limitación	Puede agrupar demasiado o demasiado poco.	Puede equivocarse por ambigüedad o cobertura limitada.

Una mayor corrección lingüística no garantiza mejores resultados de búsqueda. Compararemos su efecto con consultas y juicios de relevancia.

Fuente conceptual: Manning et al., sec. 2.2.4.

El vocabulario reúne los términos de la colección

Con una política de minúsculas y eliminación del punto final, sin stopwords ni reducción morfológica, obtenemos:

D1

`[las, tutorías, apoyan, a, estudiantes, de, primer, año]`

D3

`[la, universidad, evalúa, tutorías, para, mejorar, la, permanencia]`

$$V = \bigcup_{d \in \mathcal{D}} T(d)$$

$T(d)$ es el conjunto de términos distintos del documento d. V reúne los términos distintos de la colección y $|V|$ es su tamaño.

Fuente conceptual: Manning et al., sec. 2.2.1. Croft et al., sec. 2.2.

Las reglas elegidas cambian el tamaño del vocabulario

Usamos únicamente D1 y D3 del ejemplo anterior.

Política	Tamaño $\lvert V \rvert$
Minúsculas y separación del punto final.	14 términos
La política anterior, excluyendo además `las`, `a`, `de`, `la`, `para`.	9 términos

Entre ambos documentos hay 16 apariciones de tokens antes del filtrado. `tutorías` aparece en ambos y `la` se repite dentro de D3.

Interpretación

Reducir $\lvert V \rvert$ no demuestra una mejora de recuperación. Debemos revisar qué términos desaparecieron y cómo afecta eso a las consultas.

Documentos y consultas necesitan reglas compatibles

Documento: `Tutorías para estudiantes.`
Consulta: `TUTORIAS ESTUDIANTES`

Procesamiento	Representaciones resultantes
Documento en minúsculas, sin tildes ni stopwords.	`[tutorias, estudiantes]`
Consulta sin procesamiento.	`[TUTORIAS, ESTUDIANTES]`
Consulta con reglas compatibles.	`[tutorias, estudiantes]`

En comparación léxica exacta, diferencias introducidas por el pipeline pueden impedir una coincidencia esperada. Compatibilidad no exige que toda operación sea idéntica en ambos lados.

Fuente conceptual: Manning et al., sec. 2.2.3.

Una transformación puede perder información de forma irreversible

Texto original

`No requiere experiencia en C++.`

Supongamos que una política elimina puntuación y excluye `no` como stopword. La representación pierde la negación y la identidad del lenguaje de programación.

Diseño de la representación

Conservaremos el texto original y el ID. Guardaremos por separado la versión procesada y las reglas que la produjeron.

Esto permite mostrar evidencia fiel, revisar errores y reconstruir la representación con otra política.

Para discutir

¿Podríamos recuperar el significado original si solo conservamos los tokens modificados?

Ejercicio: dos políticas producen coincidencias diferentes

Documentos
D1: `Tutorías para estudiantes.`
D2: `Una tutoría para un estudiante.`

Consulta: `tutorías estudiantes`
Regla de selección: exigir la presencia de los dos términos procesados.

Política	Operaciones sobre documentos y consulta
A	Minúsculas, separación del punto final y exclusión de `para`, `una`, `un`.
B	La política A y lematización idealizada: `tutorías` a `tutoría`, `estudiantes` a `estudiante`.

Para discutir

¿Qué documentos selecciona cada política? ¿La coincidencia adicional prueba por sí sola que el resultado es relevante?

Interpretación del ejercicio

Política	D1	D2	Selección
A	tutorías, estudiantes	tutoría, estudiante	Solo D1
B	tutoría, estudiante	tutoría, estudiante	D1 y D2

En A, la consulta conserva los plurales. En B, la consulta también toma las formas singulares.

Lo que demuestra

La reducción morfológica amplía las coincidencias de este ejemplo. Para determinar si mejora la recuperación necesitamos conocer la necesidad y juzgar la relevancia de ambos documentos.

Las formas del ejercicio son resultados esperados bajo las reglas dadas, no una garantía sobre cualquier lematizador.

La limpieza clásica tiene un alcance específico

En esta clase diseñamos una representación para comparación léxica. Las operaciones deben respetar el método que consumirá el texto.

Representación léxica
Podemos comparar políticas de normalización, stopwords y reducción morfológica.

Modelos de lenguaje
Cuando lleguemos a embeddings, utilizaremos la tokenización del modelo y revisaremos sus requisitos de entrada.

La negación, el orden y la puntuación pueden aportar significado. Una política diseñada para un índice léxico no constituye una receta universal para cualquier modelo.

Fuente conceptual: Síntesis docente. La recuperación con embeddings corresponde a temas posteriores.

Comprobación conceptual antes de terminar

1. ¿Por qué una consulta no describe necesariamente toda la necesidad del usuario?
2. ¿Puede un documento contener todos los términos de una consulta y ser irrelevante? Proponga un caso.
3. ¿Qué diferencia hay entre contar tokens y contar términos del vocabulario?
4. ¿Qué información podríamos perder al eliminar stopwords o puntuación?
5. ¿Qué evidencia necesitaríamos para afirmar que la lematización mejoró la búsqueda?

Las respuestas deben relacionar las decisiones de representación con sus consecuencias para la recuperación.

Lecturas y referencias de esta clase

Lectura principal
Manning, C. D., Raghavan, P. y Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.

- Capítulo 1, introducción y sección 1.1: problema de RI y conceptos básicos.
- Secciones 2.1 y 2.2: unidades documentales, tokens y vocabulario.
- Sección 8.1: relevancia respecto de la necesidad de información.

Lectura complementaria
Croft, W. B., Metzler, D. y Strohman, T. (2010). *Search Engines: Information Retrieval in Practice*. Addison-Wesley. Capítulo 1 y secciones 2.1–2.2, fundamentos y arquitectura.

Los corpus breves, las actividades y los ejemplos en español son de elaboración docente. Las ilustraciones se generaron con IA y se revisaron para esta clase.